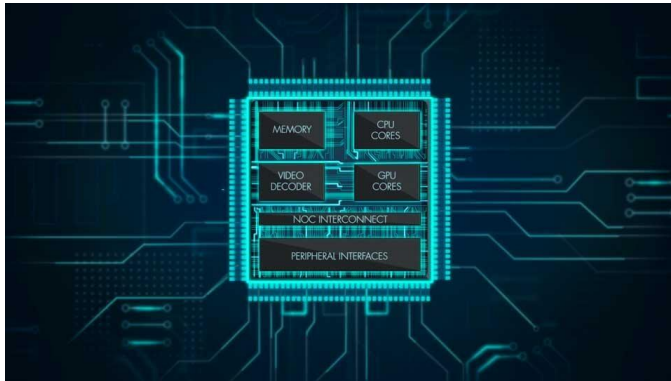


Toward Efficient Deployment of Mixture of Experts Models: Quantization and Compression Analysis



Date : 2026.01.20

Author : Dongjun Lee

Affiliation : Seoul National University of Science and Technology
Electrical and Information Engineering

Position : Master course

Contact : dj.lee@snut.ac.kr



INDEX



> I. Introduction

- 01 Mixture-of-Experts (MoE)
- 02 Limitations of MoE Models

> II. Background

- 01 Ternary Quantization
- 02 Data Compression

> III. Experimental Results

- 01 Main Result
- 02 Analysis and Discussion

> IV. Conclusion



INDEX



> I. Introduction

- 01 Mixture-of-Experts (MoE)
- 02 Limitations of MoE Models

> II. Background

- 01 Ternary Quantization
- 02 Data Compression

> III. Experimental Results

- 01 Main Result
- 02 Analysis and Discussion

> IV. Conclusion

◆ Evolution and Limitations of Transformer Models

- Transformer-based models have achieved remarkable performance across a wide range of natural language processing tasks
- Improving model performance typically requires increasing model size, dataset size, and computational resources [1]
 - State-of-the-art large language models (LLMs), such as GPT and Gemini, have been scaled to tens or even hundreds of billions of parameters
- Naively scaling model and data sizes introduces clear limitations in terms of computational cost and resource consumption
- MoE has been proposed as a new architectural paradigm that enables more efficient scaling of model capacity.

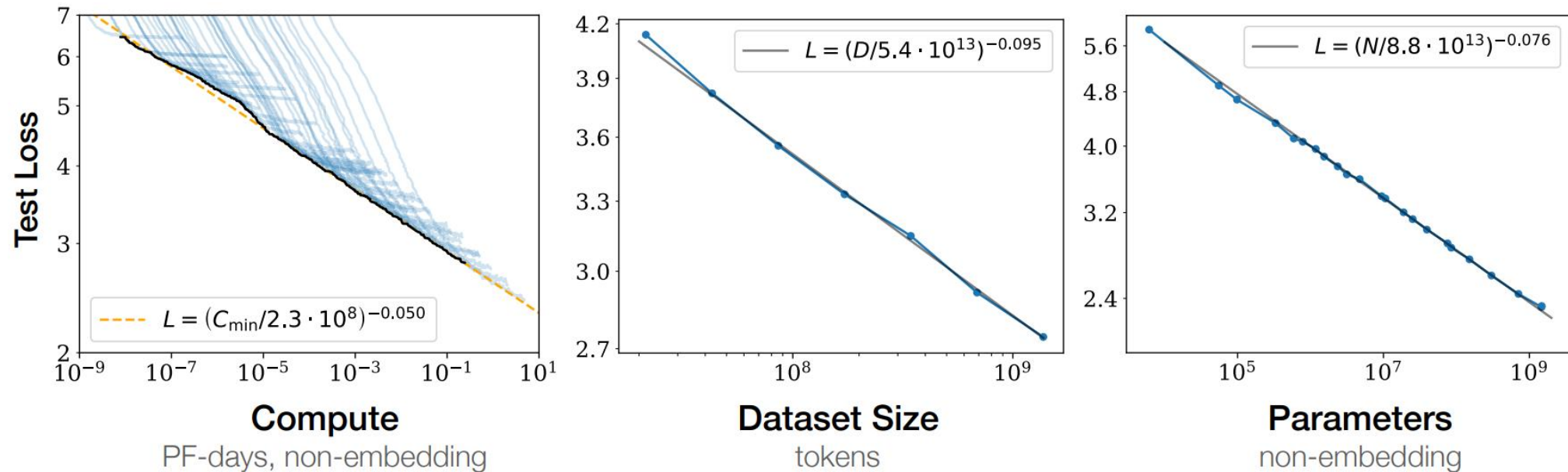


Fig. 1. Empirical scaling laws of Transformer-based language models

[1] Kaplan, Jared, et al. "Scaling laws for neural language models." arXiv preprint arXiv:2001.08361 (2020).

MoE Architecture

- The MoE architecture replaces the feed-forward network (FFN) in dense model with an MoE block composed of multiple experts
- Each MoE block contains a gating network that dynamically selects which experts to activate on a per-token basis
- During inference, tokens are routed to a subset of experts through the gating network and only the selected experts are activated
- By sparsely activating experts, MoE models achieve significantly larger model capacity while maintaining computational costs comparable to those of dense models

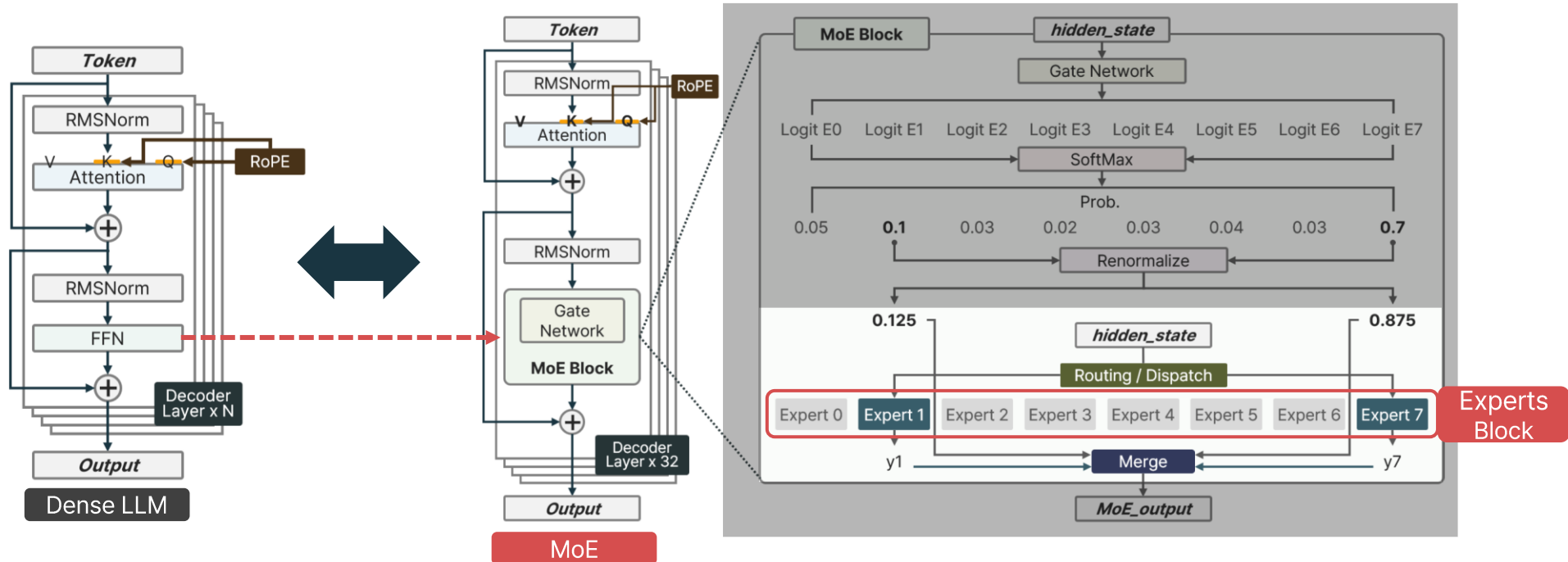


Fig. 2. Dense LLM과 MoE 모델 구조 비교 예시(Mixtral 8x7B)

Memory Bottleneck of MoE Models

- Although only a subset of experts is activated during inference, dynamic routing requires all experts to be loaded into memory
- As the number of experts increases, the total number of model parameters grows significantly, leading to a rapid increase in memory consumption
- Due to high memory requirements, MoE models are difficult to deploy on a single GPU or in memory-constrained environments
- To enable practical deployment of MoE models, additional techniques for reducing memory usage are required

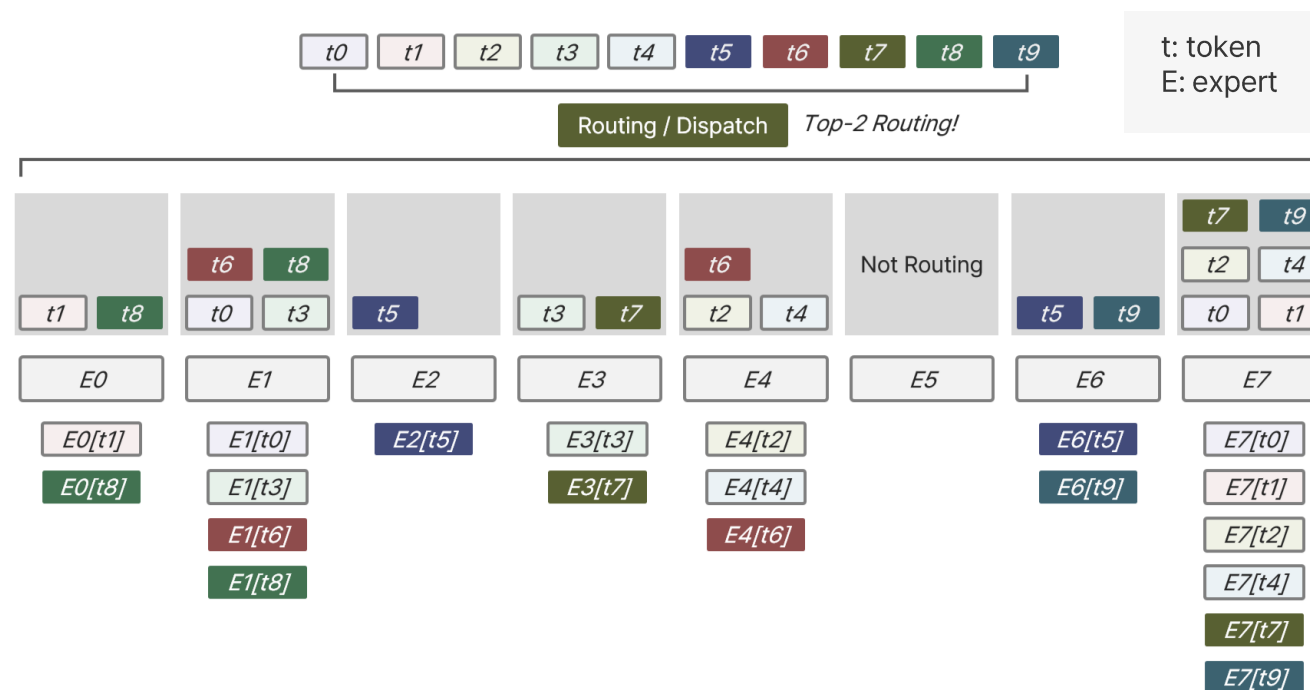


Fig. 3. Example of dynamic token routing in MoE models



INDEX



> I. Introduction

- 01 Mixture-of-Experts (MoE)
- 02 Limitations of MoE Models

> II. Background

- 01 Ternary Quantization
- 02 Data Compression

> III. Experimental Results

- 01 Main Result
- 02 Analysis and Discussion

> IV. Conclusion

Quantization

- A technique that represents data with lower-bit precision to reduce memory requirements and computational cost
- Quantization can be categorized into three types depending on the target
 - weight-only quantization
 - weight-activation quantization
 - KV cache quantization
- In MoE models, weight-only quantization is the primary focus
 - Memory requirements from expert parameters make MoE models memory-bound

Ternary Quantization

- A quantization technique that restricts weights to three discrete values
 - Weights are represented using a ternary grid of the form $\{w_{\min}, 0, w_{\max}\}$
- By reducing bit-width, model memory usage can be significantly reduced
- When ternary quantization is applied, most weights collapse to zero
 - LLM weights tend to exhibit a strong concentration around zero

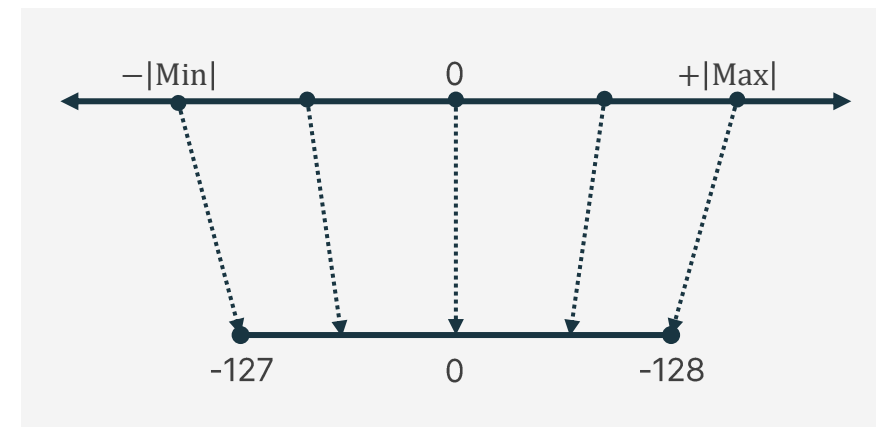


Fig. 4. Symmetric uniform quantization

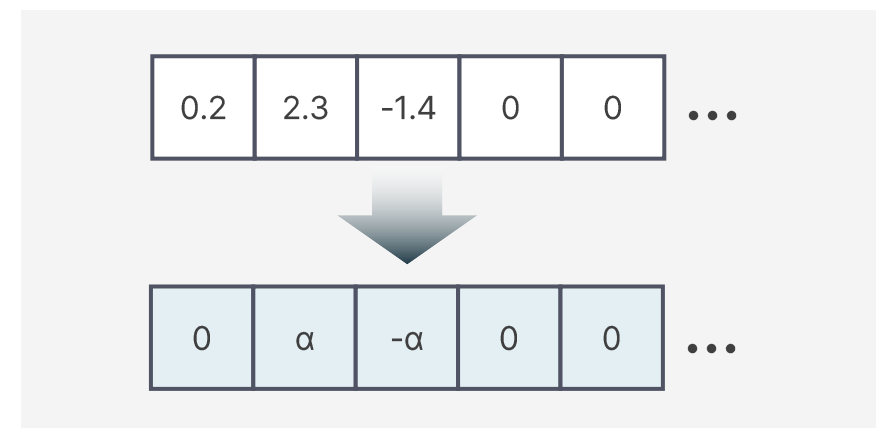


Fig. 5. Example of ternary quantization

◆ Lossless Compression

- A technique that compresses and reconstructs data without information loss, reducing overall memory usage
- Representative approaches include entropy-based coding and dictionary-based coding

◆ Entropy-Based Coding

- A method that optimizes bit representations based on the occurrence frequency of symbols
- A representative example is Huffman coding
 - Frequently occurring values are assigned shorter bit codes, while rare values are assigned longer codes
 - Due to the variable-length code property, decoding requires sequential processing

◆ Dictionary Based Coding

- A method that stores repeated data patterns in a dictionary and replaces them with indices
- A representative example is LZW-style coding
 - Repetitive structures are compressed using fixed-length indices
 - The fixed-length format enables parallel decoding



INDEX



> I. Introduction

- 01 Mixture-of-Experts (MoE)
- 02 Limitations of MoE Models

> II. Background

- 01 Ternary Quantization
- 02 Data Compression

> III. Experimental Results

- 01 Main Result
- 02 Analysis and Discussion

> IV. Conclusion

Experimental Settings

- Experiments are conducted on Switch-Transformer Base-128
- All experiments are performed using the PyTorch framework on NVIDIA A100 (80GB) GPUs
- Ternary quantization is implemented based on a prior method [2] and applied to expert weights
- Evaluation metrics:
 - Perplexity (PPL): C4 dataset, WikiText-2 dataset
 - GPU memory consumption
 - Decoding latency

Module	Embedding	Attention	Expert	Others	Total
Size (GB)	0.09	0.16	13.92	0.11	14.28

Table 1. Module-wise GPU memory consumption of the Switch-Transformer Base-128 model

Memory Distribution of Each Module

- The module-wise memory usage of the MoE model is analyzed using an FP16 baseline
- Expert modules account for approximately 97.4% of the total model memory, dominating overall memory consumption
 - The memory usage of embedding and attention modules constitutes only a negligible fraction of the total memory
- To improve the memory efficiency of MoE models, compression and optimization should primarily focus on expert modules

Quantization Result

- PPL is measured for RTN and GPTQ, with the baseline corresponding to the original model in FP16 precision
- Applying 2-bit and ternary quantization achieves more than 8× memory reduction compared to the baseline
- The GPTQ method maintains stable performance even in the ternary setting, with a limited increase in PPL

Method	W-bit	A-bit	C4 (↓)	WikiText2 (↓)
Baseline	16	16	1.73	1.97
RTN	2	16	2.27	2.43
GPTQ	2	16	1.82	1.99
RTN	1.58	16	4.54	4.27
GPTQ	1.58	16	2.39	2.27

Table 2. Perplexity (↓) comparison for different weight quantization methods

Compression on Quantized Weights

- Lossless compression is applied to ternary-quantized weights
- Dictionary-based coding
 - Achieves an 11.96× compression ratio using fixed-length indices
 - Exhibits low decoding latency of 0.72 s
- Huffman coding
 - Achieves a higher 13.49× compression ratio due to variable-length encoding
 - Incurs extremely high decoding latency of 2479.43 s

Method	Size (GB)	Comp. Ratio	Bit / Weight	Time (s)
Baseline	14.28	1.00×	16.00	–
Dictionary	1.19	11.96×	1.34	0.72
Huffman	1.06	13.49×	1.19	2479.43

Table 3. Lossless compression results on ternary-quantized expert weights

◆ Quantization Result Analysis

- Ternary quantization significantly reduces the memory requirements of expert parameters
- RTN-based quantization suffers from substantial performance degradation in extremely low-precision settings
- GPTQ maintains relatively stable performance even under ternary quantization
 - Due to their large scale, MoE models are less sensitive to quantization noise compared to smaller dense models

◆ Compression Result Analysis

- Huffman coding achieves a higher compression ratio due to its variable-length encoding scheme
 - It requires sequential decoding, resulting in prohibitively large decoding latency
- Dictionary-based coding enables efficient parallel decoding, leading to significantly lower latency
 - Since it relies on fixed-length representations, its compression ratio is lower than that of Huffman coding
- For practical deployment of MoE models, both compression ratio and decoding latency must be jointly considered



INDEX



> I. Introduction

- 01 Background
- 02 Motivation

> II. Proposed Methods

- 01 Overview
- 02 Hardware-Aware Bit-width Allocation
- 03 Mixed-Precision GEMM Orchestration

> III. Experimental Results

- 01 Experimental Settings
- 02 Main Results

> IV. Conclusion

◆ Summary

- To improve the performance of Transformer, model sizes have been continuously scaled, and MoE has been proposed as an efficient scaling paradigm
- While MoE models reduce computation through sparse routing, they suffer from severe memory overhead due to a large number of expert parameters
- We experimentally confirm that over 97% of the total memory consumption in MoE models is concentrated in expert modules
- To achieve effective MoE model compression, we investigate quantization and lossless compression techniques
 - GPTQ-based ternary quantization achieves memory savings while maintaining stable performance even under extremely low-precision settings
 - Huffman coding attains a high compression ratio via variable-length encoding, but incurs significant decoding latency
 - Dictionary-based coding uses fixed-length encoding, resulting in lower compression ratios but significantly reduced decoding latency
- For practical deployment of MoE models, system design must jointly consider compression ratio and decoding latency, rather than optimizing compression alone



Thank you!



Q & A